

Conception, développement et déploiement d'une plateforme SaaS multi-tenant pour la gestion de marketplaces bancaires avec mise en place d'un pipeline CI/CD

Rapport de Projet de Fin d'Études - Master

Sujet	Conception, développement et déploiement d'une plateforme SaaS multi-tenant pour la gestion de marketplaces bancaires avec mise en place d'un pipeline CI/CD
Organisme d'accueil	BRI Technology - informations institutionnelles à compléter et valider
Version	Document de rédaction fondé sur le code, les configurations et les captures disponibles

Note de méthode

Le présent document est une rédaction directement intégrable dans le mémoire. Elle repose sur l'analyse du code source et des artefacts réellement présents dans le dépôt Matchia : application React/Vite, API Spring Boot, PostgreSQL, Dockerfiles, fichiers Docker Compose, Jenkinsfile, configurations SonarQube, documentations métier et captures du dossier figure. Lorsque l'information institutionnelle n'est pas vérifiable dans le dépôt, elle est signalée explicitement afin d'éviter toute affirmation non fondée.

Précaution de sécurité

Certaines captures et fichiers locaux contiennent des valeurs de configuration ou des jetons. Ils ne doivent pas être reproduits dans le mémoire. Les captures proposées ci-dessous ont été sélectionnées pour leur valeur démonstrative ; toute valeur confidentielle doit être masquée avant l'impression ou la diffusion.

Chapitre 1 - Cadre général, contexte et étude de l'existant

1.1 Introduction

La transformation numérique du secteur bancaire ne se limite plus à la mise en ligne d'un catalogue institutionnel. Elle suppose la capacité de proposer des services configurables, de fédérer des partenaires, d'accompagner les clients dans leurs démarches et d'exploiter les données de manière maîtrisée. Dans ce cadre, le projet Matchia a pour ambition de mutualiser un socle applicatif tout en préservant l'identité et l'autonomie opérationnelle de chaque banque. Ce chapitre présente l'environnement général du projet, les insuffisances d'une approche fondée sur des marketplaces développées séparément, puis la réponse apportée par Matchia.

1.2 Présentation de l'organisme d'accueil

Les artefacts graphiques associés au projet identifient l'organisme d'accueil sous l'appellation BRI Technology. Le dépôt ne contient cependant ni fiche institutionnelle officielle, ni données vérifiables relatives à son effectif, à sa date de création, à ses marchés ou à son organigramme. Afin de conserver la rigueur académique attendue, ces informations doivent être complétées à partir d'une source validée par l'entreprise avant la remise finale.

Dans le cadre du présent PFE, l'organisme d'accueil constitue le contexte de réalisation d'une solution de génie logiciel orientée services financiers numériques. Le projet mobilise une architecture web moderne, des pratiques de conteneurisation, des mécanismes de sécurité applicative et des outils d'industrialisation. Cette orientation est cohérente avec une activité de conception de solutions numériques : l'équipe de réalisation doit à la fois répondre à des besoins métier, garantir l'intégration de services externes et assurer la maintenabilité de la solution livrée.

Texte à compléter avant dépôt

Insérer ici une brève présentation officielle de BRI Technology : activité, domaines d'expertise, localisation, taille ou organisation, ainsi que le rattachement exact du projet à l'équipe d'accueil. Cette complétion doit provenir d'une source institutionnelle et non être déduite du code source.

1.3 Contexte général du projet

Les banques cherchent à proposer des parcours digitaux continus, allant de la découverte d'une offre jusqu'au suivi d'une demande. Cette évolution implique la coexistence de plusieurs catégories de financement, de contenus marketing, de produits, de partenaires commerciaux et de profils utilisateurs. Lorsqu'une solution est conçue pour une seule banque, chaque nouvelle banque introduit une nouvelle variante de l'application, de ses règles de présentation et de ses données. La multiplication de ces variantes limite rapidement la capacité d'évolution du fournisseur de la solution.

Matchia répond à cette problématique par une plateforme SaaS multi-tenant. Le modèle retenu repose sur un socle applicatif commun et sur une isolation logique des données. Une banque constitue un tenant possédant sa marketplace, son identité visuelle, ses utilisateurs, ses contenus, ses stores activés, ses modules et ses produits. Les stores et les modules sont administrés comme un catalogue mutualisé, puis configurés et rendus visibles au niveau de chaque marketplace. Ainsi, l'ajout d'une banque relève principalement d'un processus de configuration et d'activation plutôt que de la création d'une nouvelle application.

Le besoin métier dépasse la mise à disposition d'une vitrine. La plateforme relie l'administrateur SaaS, qui gouverne l'écosystème, l'administrateur de banque, qui paramètre son tenant, le concessionnaire, qui gère ses produits et ses partenariats, et le client, qui consulte les offres et constitue un dossier de financement.

L'architecture prend également en compte les abonnements et les paiements en ligne, les notifications, les emails, la traçabilité et un assistant IA réservé au pilotage SaaS. Matchia se positionne ainsi comme un environnement centralisé de création, d'exploitation et d'évolution de marketplaces bancaires.

1.4 Étude de l'existant

1.4.1 Marketplaces développées séparément

Dans une approche traditionnelle, chaque banque dispose de sa propre marketplace, issue d'une copie de projet, d'une personnalisation manuelle ou d'un développement spécifique. Les couleurs, le catalogue, les contenus, les paramètres et parfois les processus sont modifiés directement dans une version dédiée du code. L'arrivée d'un nouveau client bancaire entraîne alors une phase de cadrage, de paramétrage technique, de tests et de mise en production distincte. Une correction fonctionnelle ou de sécurité doit ensuite être reportée dans plusieurs versions, avec un risque de divergence entre les installations.

Cette organisation produit une duplication de code et de configuration. Elle augmente la charge de maintenance, rallonge les délais de livraison et crée une dépendance forte à l'équipe de développement pour des opérations qui pourraient être configurables. Plus le nombre de banques augmente, plus le coût marginal d'une nouvelle marketplace et le risque de régression deviennent élevés. L'approche est donc peu adaptée à un objectif de croissance multi-banques.

1.4.2 Demande de marketplace et onboarding traditionnel

Avant l'automatisation, une banque souhaitant disposer d'une marketplace échange directement avec le fournisseur de la solution. Les besoins sont exprimés au cours de réunions ou d'échanges administratifs, puis traduits manuellement en choix de fonctionnalités, catégories et éléments de branding. La configuration est réalisée par l'équipe technique, qui doit souvent coordonner les validations, les accès et la mise en service. La banque reste dépendante de cette médiation et ne bénéficie pas d'un parcours self-service traçable.

Un tel processus limite l'industrialisation de l'onboarding. Il rend difficile le suivi uniforme des demandes, la conservation d'un historique des sélections et la liaison directe entre l'offre choisie, le paiement et l'activation des services. Il ne permet pas non plus d'offrir à l'opérateur SaaS une vision consolidée des demandes en attente, des abonnements actifs et des échéances à venir.

1.4.3 Paiement et activation dissociés

Dans un schéma non intégré, le paiement est traité en dehors de la plateforme ou selon une procédure manuelle. La vérification de la transaction, la mise à jour de l'abonnement et l'activation des services nécessitent alors des interventions complémentaires. Cette dissociation introduit des délais, des risques d'erreur de saisie et une visibilité limitée sur l'état réel du service souscrit. Elle rend également plus complexe la gestion des renouvellements et des relances d'échéance.

1.4.4 Gestion décentralisée des concessionnaires

La difficulté est particulièrement visible lorsqu'un concessionnaire travaille avec plusieurs banques. Dans un modèle de marketplaces indépendantes, le partenaire peut devoir créer un compte et gérer un catalogue dans chaque environnement. Il répète la saisie des mêmes produits, actualise plusieurs représentations de stock et suit des règles de publication distinctes. Un concessionnaire lié à trois banques peut ainsi disposer de trois espaces, de trois catalogues et de plusieurs mises à jour pour une même offre. Cette dispersion favorise les incohérences, notamment sur les informations produit et les disponibilités, tout en diminuant la lisibilité des partenariats actifs.

1.5 Problématique

Le problème à résoudre consiste à rendre la création et l'exploitation de marketplaces bancaires reproductibles, sans imposer la réalisation d'une application et d'un cycle de déploiement distincts pour chaque banque. La solution doit permettre de configurer une marketplace, ses stores, ses modules et son identité visuelle à partir d'un socle unique, tout en assurant l'étanchéité des données et des droits entre banques.

La problématique comporte également un enjeu de fluidité opérationnelle. Comment dématérialiser le parcours d'adhésion d'une banque, associer de manière fiable la validation SaaS, le paiement en ligne et l'activation du service, puis rendre l'état de l'abonnement immédiatement traçable ? Le processus attendu doit éviter les échanges physiques systématiques entre la banque et le fournisseur d'application, sans supprimer les contrôles de gouvernance nécessaires.

Enfin, la plateforme doit concilier centralisation et autonomie des partenaires. Comment offrir à un concessionnaire un compte et un catalogue uniques, réutilisables auprès de plusieurs banques, tout en laissant à chaque banque la maîtrise de ses partenariats, contrats, publications et dossiers de financement ? Cette question implique de centraliser les données de référence et le stock, mais de contextualiser leur visibilité et les décisions au niveau de chaque tenant.

Formulation synthétique

Comment concevoir, développer et déployer une plateforme SaaS multi-tenant capable de créer et de gérer des marketplaces bancaires configurables, d'automatiser l'onboarding et le paiement, de centraliser les données concessionnaires, tout en garantissant l'isolation des tenants, la sécurité et la traçabilité ?

1.6 Solution proposée : Matchia

Matchia propose de transformer la création d'une marketplace bancaire en un parcours gouverné par une plateforme unique. Le noyau SaaS conserve le catalogue commun des stores et des modules, tandis que chaque banque dispose d'une marketplace dont l'identité, les contenus, les fonctionnalités activées et les données opérationnelles sont isolés logiquement. Cette conception réduit la nécessité de copier le code et rend l'évolution fonctionnelle plus homogène.

Le parcours d'adhésion digitalise les opérations auparavant manuelles. Une banque soumet une demande, sélectionne les stores et modules correspondant à son besoin, puis sa demande est examinée par l'administrateur SaaS. Après validation, le paiement est initié via Stripe ; la confirmation côté backend déclenche l'activation de la banque, de sa marketplace, de l'abonnement et, lorsque nécessaire, du compte administrateur. Ce lien entre validation, paiement et activation garantit une cohérence qui n'existe pas dans un processus externe au système.

Le module concessionnaire centralise également le travail du partenaire. Un seul compte permet de gérer le profil, le catalogue, les documents et les stocks. Les relations avec les banques sont matérialisées par des partenariats et des contrats, puis chaque banque décide indépendamment de la publication d'un produit dans sa marketplace. Ce mécanisme préserve l'autonomie de la banque sans imposer au concessionnaire de recréer ses données dans chaque environnement.

Valeur principale

Matchia remplace une logique de développement par banque par une logique de configuration par tenant. Cette différence permet de concilier mutualisation technique, personnalisation métier, gouvernance des accès et montée en charge fonctionnelle.

1.7 Objectifs du projet

L'objectif général du projet est de concevoir et de réaliser une plateforme SaaS centralisée permettant la création et la gestion dynamique de marketplaces bancaires. Cette plateforme doit fournir un socle commun sans effacer les spécificités fonctionnelles et visuelles de chaque banque.

Objectif spécifique	Contribution de Matchia
Automatiser l'onboarding	Dépôt de demande, sélection de stores/modules, validation, paiement, activation et émission d'identifiants.
Réduire la duplication	Catalogue mutualisé et configuration de marketplace par tenant au lieu de copies de code.
Centraliser les partenaires	Compte concessionnaire unique, partenariats et contrats multi-banques, publications indépendantes.
Accompagner le client	Consultation, comparateur, simulateur, dossier de financement, pièces justificatives et suivi de décision.
Industrialiser l'exploitation	Conteneurs Docker, pipeline Jenkins, analyse SonarQube, images Docker Hub et déploiement Azure Container Apps.

Tableau 1.2 - Objectifs spécifiques du projet Matchia

1.8 Méthodologie de travail : Agile Scrum

Le projet applique la méthode Agile Scrum afin de produire la plateforme par incréments utilisables et contrôlables. Scrum est adapté à Matchia car les besoins sont interconnectés : l'isolation multi-tenant conditionne les espaces bancaires, l'activation dépend du paiement, la publication d'un produit dépend d'un partenariat, et le financement dépend des documents exigés. Au lieu de figer l'ensemble du produit avant le développement, l'équipe priorise un backlog, réalise un incrément à chaque sprint, le démontre puis ajuste les priorités selon les retours et les dépendances techniques.

1.8.1 Rôles Scrum appliqués au projet

Le Product Owner porte la vision métier : il ordonne le Product Backlog, formule les besoins sous forme de user stories et valide la valeur apportée par les incréments. Dans le contexte du PFE, ce rôle peut être assuré par le responsable métier ou le référent du projet. Le Scrum Master veille au respect du cadre Scrum, facilite la levée des obstacles et s'assure que les cérémonies restent orientées vers l'amélioration continue. L'équipe de développement conçoit, code, teste, documente et déploie l'incrément ; elle regroupe les compétences frontend, backend, base de données, qualité et DevOps nécessaires à la livraison.

1.8.2 Artéfacts et règles de préparation

Le Product Backlog rassemble les besoins classés par valeur et par dépendance : gestion des identités, tenant bancaire, catalogue de stores et modules, onboarding, paiement, espace concessionnaire, financement, assistant IA et industrialisation. Le Sprint Backlog contient les user stories sélectionnées, leurs tâches techniques et leurs critères d'acceptation. L'incrément est la partie réellement terminée et démontrable du produit. Une story est prête à être planifiée lorsqu'elle possède une description, un acteur, des critères d'acceptation, des dépendances identifiées et une estimation ; elle est terminée lorsque le code est relu, testé, intégré, documenté et déployable par le pipeline.

1.8.3 Cérémonies Scrum

Chaque sprint, de durée fixe et courte, débute par le Sprint Planning. L'équipe définit un objectif de sprint puis sélectionne les stories réalisables au regard de sa capacité. Le Daily Scrum, limité dans le temps, synchronise l'avancement, le travail à venir et les obstacles. Le Backlog Refinement prépare les besoins futurs : les stories sont clarifiées, découpées et estimées. En Sprint Review, l'incrément est présenté aux parties prenantes à partir d'un environnement de démonstration ; les retours alimentent le backlog. Enfin, la Sprint Retrospective analyse le fonctionnement de l'équipe et décide d'actions d'amélioration concrètes pour le sprint suivant.

1.8.4 Découpage incrémental de Matchia

Sprint	Objectif Scrum	Incrément démontrable
S1	Établir le socle et les accès.	Projet React/Spring Boot, PostgreSQL, authentification JWT, rôles et routes protégées.
S2	Mettre en place le noyau SaaS.	Banques tenants, marketplaces, stores, modules, contexte tenant et administration SaaS.
S3	Digitaliser l'adhésion et l'activation.	Demande bancaire, sélection de l'offre, validation SaaS, paiement Stripe et activation contrôlée.
S4	Centraliser le parcours concessionnaire.	Compte unique, produits, stock, partenariats, contrats et demandes de publication.
S5	Livrer le parcours client.	Consultation, simulation, pièces justificatives, demande de financement, décision et notifications.
S6	Fiabiliser l'exploitation.	Tests, SonarQube, Docker, Jenkins, publication des images et déploiement Azure.

Tableau 1.1 - Découpage Scrum proposé pour Matchia

Cette organisation n'empêche pas les ajustements : le Product Owner peut réordonner le backlog à la suite d'une revue, tandis que l'équipe conserve l'objectif du sprint en cours. Les critères de sécurité - contrôle des rôles, contexte tenant, validation des données et absence de secrets dans le code - font partie de la Definition of Done et ne sont pas reportés à une phase finale.

1.9 Conclusion

Ce chapitre a montré que Matchia répond à une problématique de multiplication des marketplaces bancaires, de dispersion des partenaires et de manque d'automatisation du cycle commercial. La plateforme proposée fournit un socle SaaS multi-tenant, configurable et gouverné par des règles métier. Le chapitre suivant détaille les acteurs, les exigences, les cas d'utilisation et les règles qui structurent cette solution.

Chapitre 2 - Analyse et spécification des besoins

2.1 Introduction

L'analyse des besoins a pour finalité de traduire le contexte présenté précédemment en services observables, règles contrôlables et responsabilités explicites. Elle ne se limite pas à l'inventaire des écrans : elle formalise les interactions entre les rôles, les données manipulées et les conditions de transition entre les états métier. Pour Matchia, cette étape est déterminante car la plateforme combine un périmètre SaaS transversal et plusieurs espaces opérationnels rattachés à des tenants bancaires.

2.2 Identification des acteurs

Les acteurs de Matchia sont à la fois humains et techniques. Les acteurs humains utilisent des espaces adaptés à leurs responsabilités ; les systèmes externes assurent des services spécialisés que la plateforme orchestre sans leur déléguer la gouvernance métier.

Acteur	Rôle et périmètre	Interactions principales
Administrateur SaaS	Opérateur global de la plateforme. Il dispose d'une vision transverse sur les banques, les demandes, les catalogues, les abonnements, les paiements, les audits et les demandes concessionnaires.	Valide/rejette les adhésions, administre stores/modules/marketplaces, consulte les indicateurs et interroge l'assistant IA.
Administrateur Banque	Administrateur d'un tenant bancaire. Son périmètre est limité à sa banque, sa marketplace, ses utilisateurs, ses contenus, ses produits et ses dossiers.	Paramètre branding/stores/modules, traite partenariats et publications, gère financement et abonnement.
Concessionnaire	Partenaire commercial rattaché à une catégorie de store. Il possède un espace unique, même lorsqu'il travaille avec plusieurs banques.	Soumet son dossier, gère produits/stock/documents, demande des partenariats et des publications, consulte les demandes liées à ses produits.
Client	Utilisateur inscrit dans une marketplace bancaire donnée. Il est propriétaire de son profil et de ses dossiers.	Consulte les offres, simule, téléverse les pièces, soumet et suit une demande de financement.
Internaute	Visiteur non authentifié de la plateforme ou d'une marketplace.	Consulte les contenus publics, les stores et les produits, et peut initier une inscription ou une demande d'adhésion.
Stripe	Prestataire de paiement externe.	Reçoit les demandes de Payment Intent ou Checkout Session ; son statut est vérifié avant l'activation interne.
Gemini	Service LLM externe utilisé par l'assistant SaaS.	Génère une proposition de requête et une réponse en langage naturel ; il ne reçoit aucune connexion à la base.
SMTP	Service d'envoi d'emails.	Diffuse les codes, confirmations, identifiants, liens de paiement, relances et décisions.
Jenkins / SonarQube / Docker Hub / Azure	Chaîne d'industrialisation et de livraison.	Construit, teste, analyse, conteneurise, publie et déploie les composants de Matchia.

Tableau 2.1 - Acteurs de la plateforme Matchia

2.3 Choix du modèle SaaS et de la multi-tenance

2.3.1 Modèle SaaS retenu

Le modèle Software as a Service (SaaS) consiste à proposer une application accessible en ligne, exploitée sur un socle technique commun et fournie sous forme de service. Pour Matchia, ce choix permet de remplacer des livraisons logicielles séparées par un produit unique, maintenu et déployé de façon centralisée. Les banques souscrivent une offre, sélectionnent les stores et modules nécessaires, puis exploitent une marketplace configurée selon leur identité et leurs besoins. Les mises à jour correctives et fonctionnelles sont ainsi réalisées une seule fois sur le socle, avant d'être rendues disponibles de manière maîtrisée aux tenants concernés.

2.3.2 Principe de multi-tenance

La multi-tenance permet à plusieurs organisations d'utiliser la même application sans partager leurs données opérationnelles. Dans Matchia, chaque banque représente un tenant. Son contexte détermine la marketplace consultée, les utilisateurs habilités, les contenus visibles, les produits, les dossiers de financement, les décisions et les abonnements associés. Les ressources globales, telles que le catalogue des stores et des modules, sont administrées au niveau SaaS ; leur activation et leur visibilité sont ensuite configurées pour chaque banque. L'isolation est donc logique et doit être appliquée dans les routes, les services métier, les contrôles d'autorisation et les requêtes de données.

Trois stratégies sont habituellement envisagées. Une base de données par tenant maximise l'isolation mais multiplie les opérations d'administration. Un schéma par tenant offre une séparation intermédiaire, tout en imposant une gestion spécifique des migrations. Enfin, une base et un schéma partagés avec identification du tenant mutualisent l'infrastructure et requièrent des contrôles applicatifs rigoureux. Le projet Matchia adopte cette dernière logique : les entités métier sont contextualisées par la banque et les accès sont bornés par le rôle et le tenant, ce qui correspond aux routes et aux services destinés aux espaces SaaS et bancaires.

Stratégie	Atouts	Contraintes	Position de Matchia
Base par tenant	Isolation forte et sauvegarde distincte.	Coût, supervision et migrations plus lourds.	Non retenue dans les configurations observées.
Schéma par tenant	Séparation structurelle dans une même base.	Gestion des schémas et des évolutions plus complexe.	Non retenue explicitement.
Base partagée avec tenant	Mutualisation, coûts maîtrisés, déploiement unique.	Les filtres de tenant et les autorisations doivent être systématiques.	Retenue : la banque constitue le contexte logique des données.

Tableau 2.3 - Comparaison des stratégies de multi-tenance

2.3.3 Justification du choix

Critère	Choix SaaS multi-tenant pour Matchia
Évolutivité	L'ajout d'une banque devient un processus d'adhésion, de configuration et d'activation ; il ne requiert pas une nouvelle base de code.
Mutualisation	Le frontend, le backend, les correctifs, les contrôles de sécurité et le pipeline CI/CD sont partagés et maintenus une seule fois.
Personnalisation	Chaque tenant conserve sa marketplace, son branding, ses utilisateurs, ses stores et modules activés, ses contenus et ses règles métier.
Isolation	Les droits et les données sont filtrés par tenant ; un administrateur banque ne peut accéder qu'à son périmètre bancaire.
Partenaires	Le concessionnaire centralise son catalogue et son stock, tandis que les partenariats et les publications restent contextualisés par banque.
Exploitation	Une chaîne DevOps unique construit, analyse et déploie les composants de la plateforme, ce qui réduit la répétition des opérations.

Tableau 2.2 - Justification du modèle SaaS multi-tenant

Choix d'architecture

Matchia retient une application mutualisée avec une isolation logique par banque, plutôt qu'une application et un cycle de déploiement indépendants pour chaque banque. Ce choix répond simultanément aux objectifs de centralisation, de configurabilité et de gouvernance.

2.4 Besoins fonctionnels

2.4.1 Besoins de l'administrateur SaaS

L'administrateur SaaS est responsable de la gouvernance de la plateforme. Il consulte un tableau de bord consolidé, gère les banques et les demandes d'adhésion, définit le catalogue de stores et de modules, administre les marketplaces, les contenus globaux et les utilisateurs. Il suit aussi les offres, les abonnements payés, le revenu mensuel et les échéances. La présence d'un journal d'audit, de statistiques et d'un mécanisme d'export répond au besoin de traçabilité des opérations transverses.

Cet acteur traite les demandes de création de comptes concessionnaires, accède de manière sécurisée à leurs justificatifs et déclenche la création du concessionnaire et de son administrateur lorsque le dossier est validé. Il bénéficie enfin d'un assistant IA analytique dont l'usage est explicitement restreint au rôle ADMIN_SAAS. Le système lui offre donc des fonctions CRUD, de validation, d'activation, de supervision et d'analyse, et non un simple catalogue de données.

2.4.2 Besoins de l'administrateur Banque

L'administrateur de banque pilote un tenant. Il configure l'identité de sa marketplace - couleurs, logo, bannière, texte d'accueil et contenu - et décide quels stores et modules sont activés ou visibles. Il gère les utilisateurs rattachés à sa banque, les produits bancaires et leurs paramètres, les bannières de stores et les exigences documentaires associées aux demandes de financement. Le périmètre est volontairement restreint : une banque ne doit ni administrer les données globales du SaaS ni consulter les dossiers d'un autre tenant.

L'espace bancaire prend également en charge les relations B2B. La banque peut consulter les concessionnaires disponibles pour ses stores, initier ou traiter un partenariat, gérer les contrats associés et approuver, rejeter ou inactiver les demandes de publication. Enfin, elle traite les demandes de financement liées à ses stores, consulte les pièces et communique une décision acceptée ou rejetée avec commentaire et

motif si nécessaire.

2.4.3 Besoins du concessionnaire

Le concessionnaire dépose d'abord un dossier comprenant ses données d'entreprise, un logo et des justificatifs. Après approbation, il accède à un espace unique de gestion. Il actualise son profil, consulte son dashboard, repère les banques accessibles pour son store et gère l'historique de ses partenariats. Le principe central est la réutilisation : le produit est créé une seule fois dans son catalogue, tandis que la publication est décidée séparément par chaque banque partenaire.

Le concessionnaire peut créer, modifier ou supprimer ses produits, ajuster les stocks, renseigner les caractéristiques variables définies par le store et joindre des documents. Il soumet ensuite une demande de publication liée à un partenariat actif. Il peut suivre l'état des publications, recevoir des notifications et consulter les demandes de financement concernant ses propres produits, sans accéder aux dossiers des autres concessionnaires ni aux décisions internes d'une banque.

2.4.4 Besoins du client et de l'internaute

L'internaute accède aux contenus publics de Matchia et des marketplaces : présentation, stores, produits, bannières et modules visibles. Le comparateur facilite la lecture de caractéristiques hétérogènes et le simulateur fournit une estimation à partir du montant, de l'apport, de la durée et du taux. L'inscription et l'authentification transforment ce visiteur en client rattaché à une marketplace bancaire.

Le client authentifié gère ses coordonnées, visualise un tableau de bord, crée un brouillon de demande de financement et dépose les documents exigés pour le couple banque/store. Il peut télécharger ou supprimer une pièce tant que le dossier est éditable, puis soumettre la demande lorsque toutes les pièces requises sont présentes. Il suit le statut de ses dossiers et reçoit une notification ainsi qu'un email lors d'une décision bancaire. Le simulateur n'accorde pas un crédit : il assiste la préparation du dossier, la décision demeurant une responsabilité de la banque.

2.4.5 Notifications, emails, audit et IA

Les notifications sont persistées et peuvent être marquées lues, marquées lues globalement ou supprimées. Elles couvrent notamment les demandes, les décisions, les paiements, les partenariats, les contrats, les publications et le financement. Les emails complètent ce mécanisme lors des étapes nécessitant une action hors de l'application : vérification d'adresse, identifiants temporaires, lien de paiement, rappel d'échéance, confirmation ou rejet.

Le journal d'audit conserve le contexte d'une action : tenant, acteur, rôle, type de ressource, statut, corrélation, banque ou marketplace concernée. L'assistant IA, quant à lui, n'est pas un chatbot de support générique. Il répond à des questions analytiques de l'administrateur SaaS en orchestrant une génération Text-to-SQL contrôlée ; la requête est validée, exécutée en lecture seule et bornée avant la génération de la réponse en français.

2.5 Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation global doit placer Matchia au centre et relier les cinq acteurs humains aux domaines fonctionnels qui leur sont propres. L'administrateur SaaS y est associé à la gouvernance des tenants, au catalogue, aux demandes, aux abonnements, à l'audit et à l'assistant IA. L'administrateur banque est associé à la configuration de son tenant, aux produits, aux partenaires et au financement. Le concessionnaire est associé au profil, aux partenariats, aux contrats, aux produits et aux publications. Le client est associé à l'inscription, au profil, à la simulation et au financement. L'internaute est associé à la consultation publique et au déclenchement des demandes d'inscription. Stripe, Gemini et SMTP figurent comme acteurs externes reliés respectivement au paiement, à l'assistant et aux communications.

Acteur	Cas d'utilisation principaux à relier au diagramme
SaaS	Gérer banques, demandes, marketplaces, stores, modules, utilisateurs, contenus, abonnements, paiements, concessionnaires, audits et assistant IA.
Banque	Configurer marketplace, gérer utilisateurs/contenus/produits, traiter partenaires, contrats, publications et financement.
Concessionnaire	Gérer profil, produits, stock, documents, partenariats, contrats et demandes de publication.
Client	S'inscrire, se connecter, gérer profil, simuler, créer/soumettre/consulter une demande et gérer ses documents.
Internaute	Consulter marketplace, stores et produits ; comparer, simuler, déposer une demande d'adhésion ou de concessionnaire.

Tableau 2.4 - Contenu attendu du diagramme de cas d'utilisation global

Afin de synthétiser les relations entre les acteurs et les grands domaines fonctionnels, la figure suivante propose une représentation colorée du diagramme global.

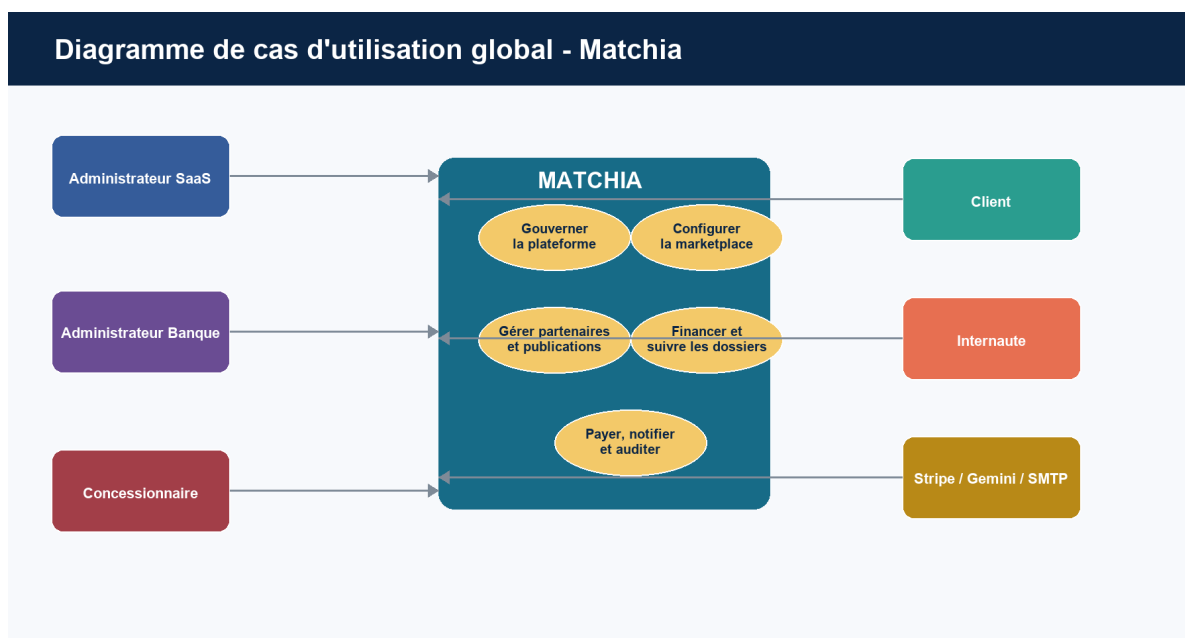


Figure 2.1 - Diagramme coloré de cas d'utilisation global de Matchia

Les couleurs distinguent les acteurs et les domaines de service. Ce diagramme doit être lu comme une vue de haut niveau : les cas d'utilisation détaillés et les contrôles métier sont précisés dans les sections suivantes.

2.6 Diagramme de classes global

Le diagramme de classes global représente les principaux concepts métier et leurs associations. La classe Bank joue le rôle de tenant : elle possède une marketplace, des utilisateurs et des abonnements. Le catalogue Store / Module est activé dans le périmètre de la marketplace. Le concessionnaire gère un ensemble de produits et un produit peut donner lieu à des demandes de financement rattachées à une banque. Ce diagramme reste volontairement conceptuel : il expose les responsabilités structurantes sans reproduire l'intégralité des attributs techniques du code.

La figure suivante synthétise les classes métier au cœur de la plateforme et rend visible la place de la banque en tant que tenant.

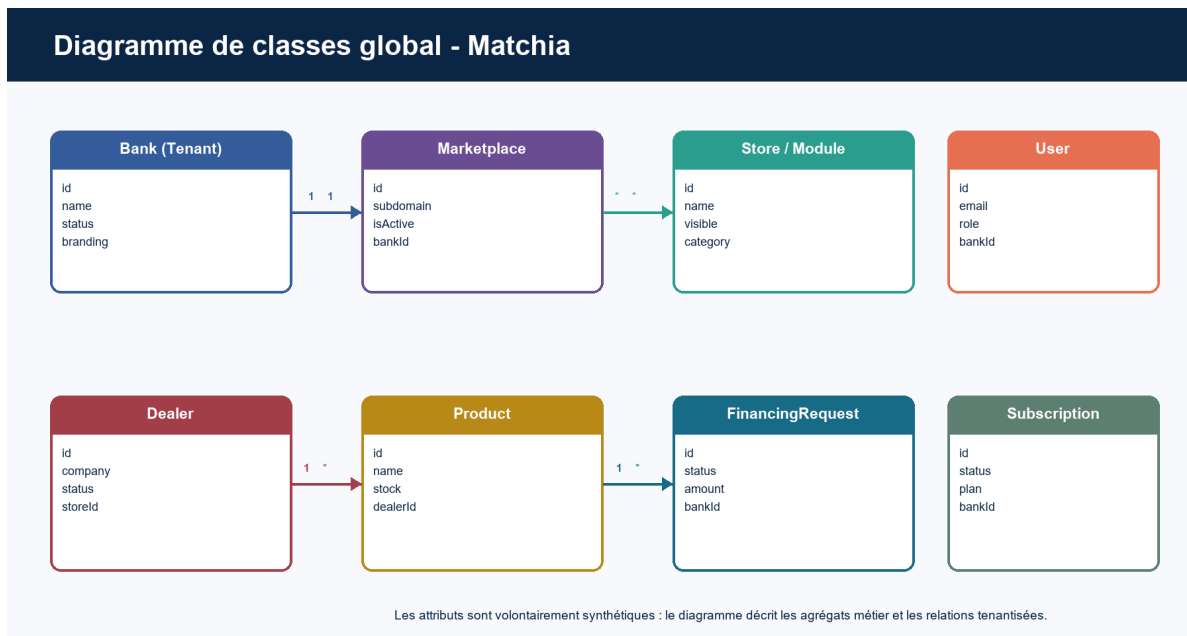


Figure 2.2 - Diagramme de classes global et tenantisé de Matchia

Les liens traduisent une lecture de haut niveau : les associations de configuration, de visibilité et les tables de liaison peuvent être détaillées dans le chapitre de conception. Le point essentiel est que les ressources opérationnelles sont toujours rattachées ou contextualisées par le tenant concerné.

2.7 Diagramme d'activité : traitement d'une demande de financement

Le diagramme d'activité décrit le comportement du système depuis la préparation du dossier jusqu'à la décision bancaire. Il met en évidence les points de contrôle : complétude des pièces, traitement par la banque du tenant et obligation d'un motif en cas de rejet. La réservation de stock n'intervient qu'après une décision favorable lorsque le produit est associé à un concessionnaire.

Le flux suivant formalise le cycle de vie d'une demande de financement au sein d'une marketplace bancaire.

Diagramme d'activité - Demande de financement

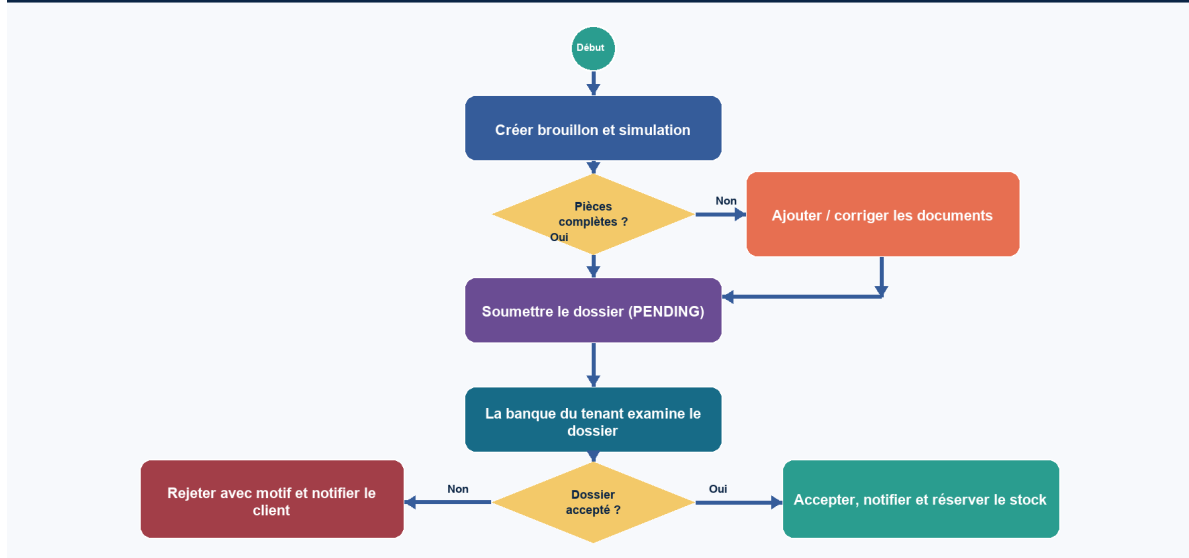


Figure 2.3 - Diagramme d'activité coloré d'une demande de financement

Les boucles de correction empêchent la soumission d'un dossier incomplet. Les deux issues finales assurent une communication explicite au client, tout en maintenant l'isolation des décisions au niveau de la banque concernée.

2.8 Cas d'utilisation détaillés

UC1 - Adhésion d'une banque

Élément	Description
Acteur principal	Internaute représentant une banque
Préconditions	Le formulaire d'adhésion est accessible ; l'adresse email est vérifiée lorsque le parcours le requiert.
Scénario nominal	Le représentant renseigne les informations de la banque, sélectionne stores et modules, puis soumet la demande. Le SaaS reçoit un dossier pending et le contact reçoit une confirmation.
Scénarios alternatifs	Email non vérifié, données invalides, sélection incomplète ou demande rejetée avec motif.
Postconditions	Une demande historisée, associée aux sélections et prête au traitement SaaS, est créée.

UC2 - Validation, paiement et activation

Élément	Description
Acteur principal	Administrateur SaaS puis contact bancaire
Préconditions	Une demande d'adhésion est en attente et comporte les informations requises.
Scénario nominal	Le SaaS approuve la demande ; le contact utilise le parcours Stripe ; le backend confirme le paiement et active banque, marketplace, abonnement et compte.
Scénarios alternatifs	Rejet motivé, paiement annulé ou statut Stripe non payé : l'activation ne doit pas être réalisée.
Postconditions	Les composants concernés sont actifs seulement après confirmation de paiement.

UC3 - Partenariat et publication dealer

Élément	Description
Acteur principal	Concessionnaire et administrateur Banque
Préconditions	Le dealer est actif ; le store est compatible ; la banque est disponible.
Scénario nominal	Le dealer demande un partenariat ; la banque le traite ; un contrat est envoyé et accepté ; le dealer soumet un produit ; la banque décide la publication.
Scénarios alternatifs	Doublon dealer-banque-store, rejet, suspension, contrat expiré ou produit inactif.
Postconditions	Le produit n'est visible publiquement que si la publication et les dépendances restent actives.

UC4 - Demande de financement

Élément	Description
Acteur principal	Client puis administrateur Banque
Préconditions	Le client est authentifié dans un tenant ; le store et le produit sont accessibles.
Scénario nominal	Le client crée un brouillon, renseigne la simulation, dépose les pièces requises et soumet. La banque consulte le dossier et rend une décision.
Scénarios alternatifs	Pièce manquante, produit hors tenant, dossier non brouillon, rejet sans motif ou tentative d'accès à un dossier non autorisé.
Postconditions	Le dossier passe à ACCEPTED ou REJECTED ; le client est notifié et, pour un produit dealer accepté, le stock est réservé.

UC5 - Interrogation IA contrôlée

Élément	Description
Acteur principal	Administrateur SaaS
Préconditions	L'utilisateur dispose du rôle ADMIN_SAAS et la configuration Gemini est disponible.
Scénario nominal	La question est envoyée au backend ; le schéma filtré et le contexte sont transmis au LLM ; le SQL proposé est validé, exécuté en lecture seule et reformulé.
Scénarios alternatifs	SQL non conforme, donnée absente, erreur d'exécution ou dépassement des règles de sécurité.
Postconditions	La réponse française est retournée sans exposition des champs ou identifiants sensibles.

2.9 Diagrammes de séquence recommandés

Les diagrammes de séquence suivants doivent être insérés dans les chapitres de conception et de réalisation. Ils permettent de rendre visibles les responsabilités du frontend, de l'API, de la base et des services externes. Leur objectif est de montrer les contrôles, non de reproduire les détails du code.

Diagramme	Participants	Étapes à représenter
Authentification	Utilisateur, React, API Auth, filtre JWT, base.	Login, émission access/refresh token, requête protégée, renouvellement et refus d'accès.
Onboarding banque	Représentant, frontend, RequestService, SaaS, base, SMTP.	Dépôt, vérification, sélection, persistance, notification et validation/rejet.

Diagramme	Participants	Étapes à représenter
Paielement et activation	Contact banque, frontend, PaymentService, Stripe, base, SMTP.	Création session/intention, confirmation Stripe, mise à jour état, activation et email.
Partenariat/pu blication	Dealer, banque, services partenariat/contrat/produit, base, notification.	Demande, décision, contrat, soumission produit, approbation/rejet de publication.
Financement	Client, API, base, banque, dealer éventuel, SMTP.	Brouillon, documents, contrôle, soumission, décision, notification et réservation stock.
Assistant IA	Admin SaaS, widget, API IA, Gemini, validateur SQL, PostgreSQL.	Question, schéma filtré, SQL proposé, validation, lecture bornée, reformulation ou rejet.

Tableau 2.5 - Diagrammes de séquence à réaliser

2.10 Workflows métier colorés

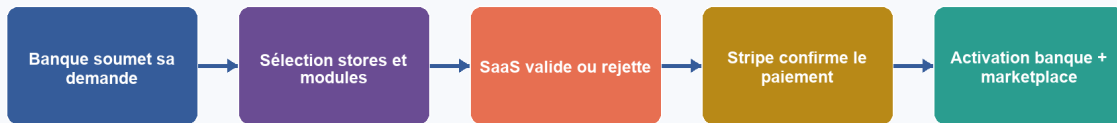
Le rapport de conception peut compléter les diagrammes présentés ci-dessous par des diagrammes UML ciblés : séquence d'authentification, séquence d'onboarding bancaire, séquence de paiement, séquence de partenariat banque-concessionnaire, séquence de financement et séquence de l'assistant IA. Un diagramme de composants est pertinent dans le chapitre de conception pour relier frontend React, API Spring Boot, PostgreSQL et services externes. Enfin, le diagramme de déploiement présenté au chapitre 6 porte l'architecture cloud réelle : frontend et backend conteneurisés, images Docker Hub et Azure Container Apps.

Diagramme	Objectif	Emplacement recommandé
Cas d'utilisation global	Délimiter les acteurs et les services majeurs.	Section 2.5.
Classes global	Présenter les entités métier et les associations tenantisées.	Section 2.6.
Activité	Décrire les décisions et les états du financement.	Section 2.7.
Séquence	Détailler les échanges frontend, backend et services externes.	Section 2.9 / chapitre de conception.
Composants et déploiement	Présenter les composants techniques et l'hébergement cloud.	Chapitre de conception et section 6.11.

Tableau 2.6 - Diagrammes UML et emplacements recommandés

Le parcours commercial de Matchia associe la validation de la demande au paiement puis à l'activation. La figure suivante formalise ce séquençement.

Onboarding bancaire : validation, paiement et activation



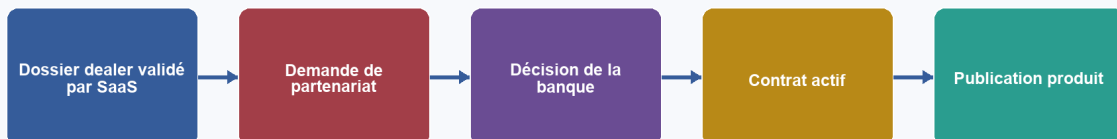
Si la demande est rejetée ou si le paiement n'est pas confirmé, l'activation n'est pas déclenchée.

Figure 2.4 - Diagramme coloré du parcours d'onboarding et de paiement

L'activation n'est jamais une simple conséquence de l'envoi du formulaire. Elle dépend de la validation SaaS et de la confirmation de paiement vérifiée par le backend.

Le schéma ci-dessous rend visible le cycle B2B qui relie la création du partenaire, le contrat et la publication du produit.

Cycle concessionnaire : partenariat, contrat et publication



La visibilité publique exige que le concessionnaire, le partenariat, le contrat, le store et la marketplace restent actifs.

Figure 2.5 - Diagramme coloré du cycle concessionnaire

La publication est une décision indépendante de chaque banque. Cette indépendance permet au concessionnaire de centraliser son catalogue sans retirer à la banque son contrôle sur les offres visibles dans sa marketplace.

Le workflow de financement est présenté sous forme d'étapes ordonnées, depuis la simulation jusqu'à la communication de la décision.

Workflow de demande de financement

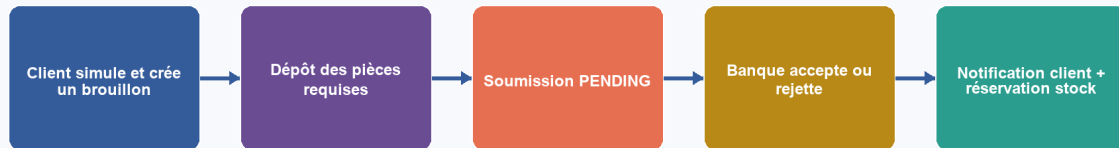


Figure 2.6 - Diagramme coloré du workflow de financement

La séparation entre brouillon, soumission et décision garantit que les documents obligatoires sont contrôlés avant le traitement bancaire. Le processus préserve également l'isolation du tenant concerné.

Le dernier diagramme de la présente analyse illustre la chaîne de sécurité de l'assistant IA réservé à l'administration SaaS.

Assistant IA : Text-to-SQL sécurisé et lecture seule



Figure 2.7 - Diagramme coloré de l'assistant IA sécurisé

Gemini ne se connecte jamais directement à PostgreSQL. Le validateur backend impose une allowlist, une lecture seule, un timeout et une borne de résultats avant la reformulation de la réponse.

2.11 Exigences non fonctionnelles

Exigence	Mécanismes présents dans Matchia	Critère de validation
Sécurité	JWT stateless, refresh token, rôles, contrôle de tenant dans les services, validation DTO, CORS, fichiers protégés.	Un utilisateur non autorisé ne consulte ni ne modifie une ressource hors de son périmètre.
Confidentialité	Relations banque/client/dealer, téléchargement contrôlé, filtrage IA des colonnes sensibles.	Un dossier ou document est récupéré uniquement par son propriétaire ou une banque habilitée.
Intégrité	Contraintes d'unicité, statuts énumérés, transactions JPA, version optimiste sur financement.	Les doublons et transitions d'état non autorisées sont refusés.
Performance	API REST, index de demandes, filtres, limitation de résultats IA à 50 et timeout de 15 secondes.	Les consultations ne chargent pas de données inutilement volumineuses.
Scalabilité	Tenant bancaire logique, catalogues réutilisables, conteneurs indépendants frontend/backend.	Une banque peut être ajoutée par configuration sans clone applicatif.
Maintenabilité	Séparation React/Spring, couches controller-service-repository, DTO/mappers, TypeScript, tests et SonarQube.	Une évolution métier peut être localisée et vérifiée par les outils de qualité.
Disponibilité	Docker Compose local, images versionnées, Azure Container Apps et procédures de redéploiement.	Les services peuvent être reconstruits et redéployés de manière reproductible.

Tableau 2.7 - Exigences non fonctionnelles

2.12 Règles métier

Les règles métier sont principalement appliquées dans les services Spring. Elles constituent une seconde ligne de défense après les restrictions de navigation et de rôle du frontend. Les règles essentielles sont les suivantes.

Référence	Règle formalisée
RM1	Une banque et sa marketplace ne deviennent actives qu'après le traitement favorable de la demande et la confirmation d'un paiement payé.
RM2	Un store ou un module apparaît dans une marketplace seulement lorsqu'il est affecté, activé et visible pour ce tenant.
RM3	Une demande de financement client doit concerner un store actif sur la marketplace du client et un produit cohérent avec ce store et ce tenant.
RM4	Un dossier de financement ne peut être soumis qu'à l'état DRAFT et après dépôt de toutes les pièces obligatoires actives.
RM5	Une banque ne traite que les dossiers de financement de son propre tenant ; le rejet exige un motif.
RM6	Le concessionnaire n'accède qu'à son profil, ses produits, ses partenariats, ses contrats, ses publications et aux dossiers liés à ses produits.
RM7	Le triplet concessionnaire-banque-store est unique pour empêcher des demandes de partenariat concurrentes ou dupliquées.
RM8	Une publication de produit est décidée indépendamment par chaque banque et ne devient publique que si produit, concessionnaire, partenariat, store et marketplace sont actifs.
RM9	Un stock de produit concessionnaire est réservé lors de l'acceptation d'une demande de financement portant sur ce produit.
RM10	L'assistant IA ne peut exécuter qu'une requête SELECT validée, bornée et en lecture seule ; les champs sensibles sont exclus.

Tableau 2.8 - Principales règles métier de Matchia

2.13 Backlog produit, releases et sprints

La chronologie proposée suit les dépendances techniques observées dans le projet. Le socle d'identité précède la configuration multi-tenant ; le paiement s'appuie sur les demandes et abonnements ; le financement s'appuie sur les produits et le contexte de marketplace ; l'assistant IA et le DevOps consolident ensuite l'ensemble. Les sprints doivent être adaptés à la durée réelle du PFE, mais cet ordre donne une lecture cohérente de l'évolution du produit.

Release	Objectif	Livrable et fonctionnalités
R1 - S1/S2	Construire le socle sécurisé.	Modèle Bank/User, authentification JWT/refresh, rôles, profils, routes protégées, PostgreSQL et bases stores/modules.
R2 - S3/S4	Instancier les tenants bancaires.	Demandes d'adhésion, vérification email, validation/rejet, marketplace, slug, branding, stores/modules et contenus par banque.
R3 - S5	Rendre la marketplace exploitable.	Accueil tenant, stores, produits bancaires, paramètres, bannières, comparateur, simulateur et blog.
R4 - S6	Automatiser le cycle commercial.	Offres, abonnements, Stripe, paiement, activation, notifications, emails, renouvellement et revenus.
R5 - S7/S8	Intégrer les concessionnaires.	Dossier dealer, partenariats, contrats, catalogue unique, stock, documents et publications multi-banques.
R6 - S9	Gérer le financement client.	Inscription tenant, profil, dossier, pièces, soumission, décision bancaire, suivi dealer et réservation.
R7 - S10	Apporter analyse et traçabilité.	Audit, tableaux de bord et assistant IA Text-to-SQL sécurisé.
R8 - S11/S12	Industrialiser la livraison.	Tests, JaCoCo/Vitest, SonarQube, Docker, Jenkins, Docker Hub, Azure et recette.

Tableau 2.9 - Proposition de releases et sprints

2.14 Conclusion

L'analyse des besoins confirme que Matchia est un système multi-acteurs et multi-processus. Les exigences combinent gouvernance SaaS, autonomie contrôlée des banques, centralisation des concessionnaires, parcours client documenté et intégrations externes. Les règles métier garantissent que la configurabilité n'affaiblit ni l'isolation des tenants ni la cohérence des transitions. Ces spécifications constituent la base de l'architecture et de la réalisation présentées dans les chapitres suivants du mémoire.

Chapitre 6 - DevOps et déploiement

6.1 Introduction

Le développement d'une plateforme SaaS ne s'arrête pas à la production du code applicatif. Sa qualité et sa disponibilité dépendent de la capacité à construire les composants de manière reproductible, à exécuter les contrôles nécessaires, à produire des images cohérentes et à déployer une version identifiée. Matchia met en œuvre une chaîne DevOps associant Git/GitHub, Jenkins, Maven, Node.js, SonarQube, Docker, Docker Hub et Azure Container Apps. Ce chapitre décrit exclusivement les mécanismes vérifiables dans les fichiers de configuration et les captures présentes dans le dépôt.

6.2 Environnements du projet

Environnement	Rôle	Éléments vérifiés
Développement local	Codage, exécution et recette fonctionnelle sur poste de travail.	React/Vite, Spring Boot sur le port 8081, PostgreSQL local ou externe, uploads locaux.
Docker local	Reproduction de l'exécution applicative dans deux conteneurs.	Compose principal, images matchia-frontend et matchia-backend, réseau bridge, volume uploads.
Outils CI/qualité	Automatisation des builds et analyse statique.	Jenkins containerisé avec Docker-in-Docker ; SonarQube et PostgreSQL Sonar dans un Compose dédié.
Cloud	Mise à disposition des images de production.	Jenkinsfile : Docker Hub puis az containerapp update pour frontend et backend dans Azure Container Apps.

Tableau 6.1 - Environnements identifiés dans le projet

6.3 Gestion du code source avec Git et GitHub

Le Jenkinsfile configure un déclencheur githubPush() et commence par une étape Checkout exécutée sur le dépôt associé au job. Le cycle de livraison est donc initié par une modification versionnée : le développeur réalise ses changements, les intègre au dépôt GitHub, puis Jenkins récupère l'état correspondant pour lancer les stages. Cette organisation apporte une traçabilité entre la version source, le numéro de build Jenkins et les images Docker publiées.

Le dépôt contient également une capture liée au paramétrage de Jenkins et une capture de création de pipeline. Ces artefacts démontrent l'existence d'un pipeline déclaré dans l'outil. Les détails de gouvernance Git - conventions de branches, stratégie de revue ou règles de fusion - ne sont pas documentés de manière fiable dans les fichiers disponibles ; ils ne doivent donc pas être présentés comme des pratiques formalisées sans validation de l'équipe.

6.4 Conteneurisation avec Docker

6.4.1 Image backend

Le Dockerfile du backend adopte une construction multi-stage. La première image repose sur Maven 3.9 et Eclipse Temurin 17. Le fichier pom.xml est copié avant le code source afin de télécharger les dépendances avec mvn dependency:go-offline, puis le répertoire src est ajouté et la commande mvn clean package -DskipTests génère le JAR. La seconde image, Eclipse Temurin 17 JRE, ne conserve que le JAR produit et démarre l'application avec java -jar app.jar. Cette séparation réduit la surface de l'image d'exécution en évitant d'y embarquer Maven et les fichiers source.

Le port 8081 est exposé conformément à la configuration Spring Boot. Les tests ne sont pas exécutés dans ce Dockerfile ; ils sont séparés de la phase de conteneurisation et pris en charge par l'étape Tests Backend du pipeline Jenkins. Cette séparation doit être explicitée : une image construite avec -DskipTests n'est acceptable que si la chaîne CI exécute les tests avant la publication de l'image.

6.4.2 Image frontend

Le Dockerfile du frontend suit également deux étapes. Une image Node 22 Alpine copie les descripteurs de dépendances, installe les paquets, récupère le code et lance npm run build. La variable VITE_API_URL est passée comme argument de build puis injectée dans l'environnement de compilation Vite. La seconde étape s'appuie sur Nginx Alpine, remplace la configuration par défaut, copie le contenu du dossier dist et expose le port 80. La règle try_files de Nginx redirige les routes de la SPA vers index.html, ce qui permet de conserver le routage React après un rafraîchissement de page.

La phase de construction produit une image distincte pour le frontend et pour le backend. La capture suivante est retenue car elle montre le résultat des deux builds sans exposer de valeur confidentielle.

```
[+] build 2/2
✓Image matchia-frontend Built 2.2s
✓Image matchia-backend Built 2.2s
```

Figure 6.1 - Construction réussie des images Docker Matchia

Cette figure confirme que les deux artefacts applicatifs sont produits séparément. Cette indépendance facilite l'évolution d'une couche sans imposer la reconstruction de l'autre en dehors du pipeline qui les coordonne.

6.4.3 Base de données

La base métier de Matchia est PostgreSQL. Dans le fichier docker-compose.yml principal, elle n'est pas déclarée comme un service Docker : le backend utilise une URL JDBC fournie par variables d'environnement et, pour l'environnement local décrit, l'hôte host.docker.internal permet au conteneur d'atteindre PostgreSQL installé hors du réseau Compose. Cette précision est importante : le mémoire ne doit pas représenter un troisième conteneur PostgreSQL dans le Compose principal alors qu'il n'est pas présent dans ce fichier.

Un PostgreSQL distinct est en revanche défini dans docker-compose.sonar.yml pour stocker les données propres à SonarQube. La séparation entre la base métier et la base d'outillage évite de confondre les données fonctionnelles de Matchia avec les métriques de qualité logicielle.

6.5 Orchestration avec Docker Compose

Le Compose principal orchestre deux services : backend et frontend. Le backend est construit à partir de MatchiaBackend/Dockerfile, publie le port 8081 et monte le répertoire uploads afin de préserver les fichiers générés. Le frontend est construit à partir de MatchiaFrontend/Dockerfile, reçoit l'URL API au build, publie le port 5173 vers le port 80 Nginx et dépend du backend. Les deux services sont attachés au réseau bridge matchia-network. Le mode restart unless-stopped traduit une volonté de reprise automatique après un redémarrage de l'hôte Docker, sans prétendre constituer à lui seul une stratégie de haute disponibilité.

Deux autres fichiers Compose complètent cette architecture. docker-compose.jenkins.yml exécute Jenkins avec un service Docker-in-Docker, des certificats TLS et des volumes persistants. docker-compose.sonar.yml exécute SonarQube Community avec une base PostgreSQL dédiée, un healthcheck et des volumes pour les données, extensions et journaux. Cette organisation sépare l'exécution métier de l'outillage de build et de qualité.

Après la construction des images, le démarrage local est réalisé par Docker Compose. La capture ci-dessous montre le lancement coordonné des deux conteneurs applicatifs.

```
PS D:\PFE M2\Plateforme SaaS\MatchiaFrontend> docker compose up -d
[+] up 2/2
✓Container matchia-backend Started 0.5s
✓Container matchia-frontend Started 0.7s
PS D:\PFE M2\Plateforme SaaS\MatchiaFrontend> |
```

Figure 6.2 - Démarrage des conteneurs applicatifs via Docker Compose

La sortie atteste que matchia-backend et matchia-frontend ont été démarrés. Elle illustre la reproductibilité de l'environnement local, tout en rappelant que la base métier reste un service externe dans cette configuration.

6.6 Intégration continue avec Jenkins

Le Jenkinsfile définit un pipeline déclaratif. Il désactive le checkout implicite afin de rendre explicite la récupération du code dans le stage Checkout, puis configure githubPush() comme déclencheur. Une variable AZURE_BACKEND_URL contient l'URL du backend déployé ; elle est utilisée lors de la construction du frontend pour produire une version qui pointe vers l'API Azure. Le pipeline est constitué de stages qui réalisent la construction, les tests backend, les analyses SonarQube, la création d'images, leur publication sur Docker Hub et la mise à jour des deux Container Apps.

Stage Jenkins	Commande ou mécanisme vérifié	Résultat attendu
Checkout	checkout scm	Récupération de la révision GitHub associée au build.
Build Backend	./mvnw clean package -DskipTests	Compilation Spring Boot et génération du JAR.
Build Frontend	npm ci puis VITE_API_URL=... npm run build	Installation déterministe et production de dist.
Tests Backend	./mvnw test	Exécution des tests Java et génération du rapport JaCoCo configuré dans Maven.
Sonar Backend	sonar-maven-plugin avec rapport jacoco.xml	Analyse statique du backend et remontée de couverture.
Sonar Frontend	npx @sonar/scan avec token Jenkins	Analyse du code TypeScript/React et du rapport lcov.
Docker Build	docker build backend et frontend	Création d'images locales version de build.
Push Docker Images	tag build/latest puis docker push	Publication des deux images sur Docker Hub.
Deploy Azure	az containerapp update frontend et backend	Création d'une nouvelle révision des Container Apps.

Tableau 6.2 - Stages réellement définis dans le Jenkinsfile

La capture de Jenkins synthétise les stages exécutés lors d'un build réussi. Elle est particulièrement pertinente car sa séquence correspond aux étapes déclarées dans le Jenkinsfile.

```

#8 exporting to image
#8 exporting layers
#8 exporting layers 5.1s done
#8 exporting manifest sha256:355d2cd73ab29e86afe56e8f4a1eae44d839fe69416472eed1f2d618a2e5f2cc 0.0s done
#8 exporting config sha256:a7008e1c9145f9babf10cab4aaa36d9591dd1309d377b44078ecb42ebd4aaffe 0.0s done
#8 exporting attestation manifest sha256:b896eef08abb7594dc205cfed2ea9df2c235d1477ced979f9ad618f5004d17d 0.0s done
#8 exporting manifest list sha256:a752a2ab0d94c5adb4f0e0f1935aedeaed00a5e6cca729363ab610d84e330c1b 0.0s done
#8 naming to docker.io/library/matchia-jenkins:latest
#8 naming to docker.io/library/matchia-jenkins:latest done
#8 unpacking to docker.io/library/matchia-jenkins:latest
#8 unpacking to docker.io/library/matchia-jenkins:latest 0.8s done
#8 DONE 6.1s

#9 resolving provenance for metadata file
#9 DONE 0.0s
[+] build 1/1
✓Image matchia-jenkins:latest Built

```

Figure 6.3 - Exécution réussie du pipeline Matchia sous Jenkins

La figure montre le checkout, les builds backend et frontend, les tests backend, les analyses SonarQube, la construction et la publication Docker, puis les deux déploiements Azure. Elle constitue une preuve visuelle du déroulement de la chaîne CI/CD.

Limite observée

Le Jenkinsfile ne contient pas de stage explicite waitForQualityGate. SonarQube est exécuté et une capture montre un Quality Gate validé, mais l'arrêt automatique du pipeline en attente du verdict du Quality Gate n'est pas visible dans le script. Cette étape doit être présentée comme une amélioration recommandée, non comme un verrou déjà codé.

6.7 Build du backend et 6.8 Build du frontend

Le backend est construit avec le Maven Wrapper présent dans MatchiaBackend. Le pipeline exécute d'abord `clean package -DskipTests` afin de produire l'artefact de déploiement, puis `./mvnw test` dans un stage distinct. Le `pom.xml` déclare Java 17, Spring Boot, JPA, Security, Validation, Mail, PostgreSQL, Stripe, WebFlux, JWT, les starters de test et JaCoCo. Le rapport XML généré par JaCoCo est ensuite référencé par l'analyse Sonar Maven.

Le frontend est construit avec Node.js 24 dans Jenkins. Le stage utilise `npm ci`, commande adaptée à `package-lock.json` et à une installation reproductible, puis `npm run build`. Ce script enchaîne la vérification TypeScript `tsc -b` et le build Vite. Le `package.json` définit également `npm run test` et `npm run test:coverage`, mais ces commandes ne sont pas appelées dans le Jenkinsfile courant. L'ajout d'un stage Tests Frontend avant l'analyse SonarQube constitue donc une amélioration prioritaire de la chaîne.

6.9 Tests automatisés

Le code source contient des tests backend dans MatchiaBackend/src/test et des tests frontend Vitest dans MatchiaFrontend/src. Le backend couvre notamment les services métier, les contrôles de sécurité, les transitions de financement et les services dealer. La configuration Maven intègre les dépendances de test Spring ainsi que Testcontainers pour PostgreSQL. Le frontend dispose de tests de pages publiques, services API, utilitaires de tenant, visibilité des modules, comparaison, stockage de session et assistant IA. Les rapports de couverture frontend existants utilisent le format `lcov`, conformément à `sonar-project.properties`.

Le nombre de fichiers de test ne constitue pas à lui seul une preuve de qualité. Dans le mémoire, il convient de présenter les résultats réellement produits à la date de livraison - succès/échec, taux de couverture et périmètre - sans inventer de pourcentage. La capture SonarQube backend disponible affiche un Quality Gate validé mais une couverture de 0,0 % pour l'analyse capturée ; cette valeur doit être interprétée comme un état de l'outil à cet instant et non comme une couverture définitive du projet.

6.10 Analyse de qualité avec SonarQube

SonarQube est configuré pour les deux couches. Côté backend, le Jenkinsfile appelle le plugin sonar-maven-plugin et lui indique le chemin du rapport JaCoCo. Côté frontend, sonar-project.properties définit la clé du projet, les sources src, les exclusions de dépendances, de build, d'images et de composants UI, les tests .test.ts/.test.tsx ainsi que coverage/lcov.info. Le stage Jenkins utilise @sonar/scan avec les variables d'environnement de l'installation SonarQube et un credential dédié.

L'objectif de cette analyse est de rendre visibles les problèmes de fiabilité, maintenabilité, sécurité, duplication et couverture. Elle donne à l'équipe un indicateur commun avant la publication d'une image. SonarQube Community possède toutefois des limites de couverture fonctionnelle ; l'analyse statique doit être complétée par les tests applicatifs, les revues de code et les contrôles de secrets. Il est notamment indispensable de ne jamais conserver un token dans une capture ou dans un fichier versionné.

La figure suivante est retenue pour illustrer les indicateurs de qualité du backend. Les valeurs affichées correspondent à l'analyse visible dans l'environnement SonarQube et doivent être datées dans la version finale du mémoire.

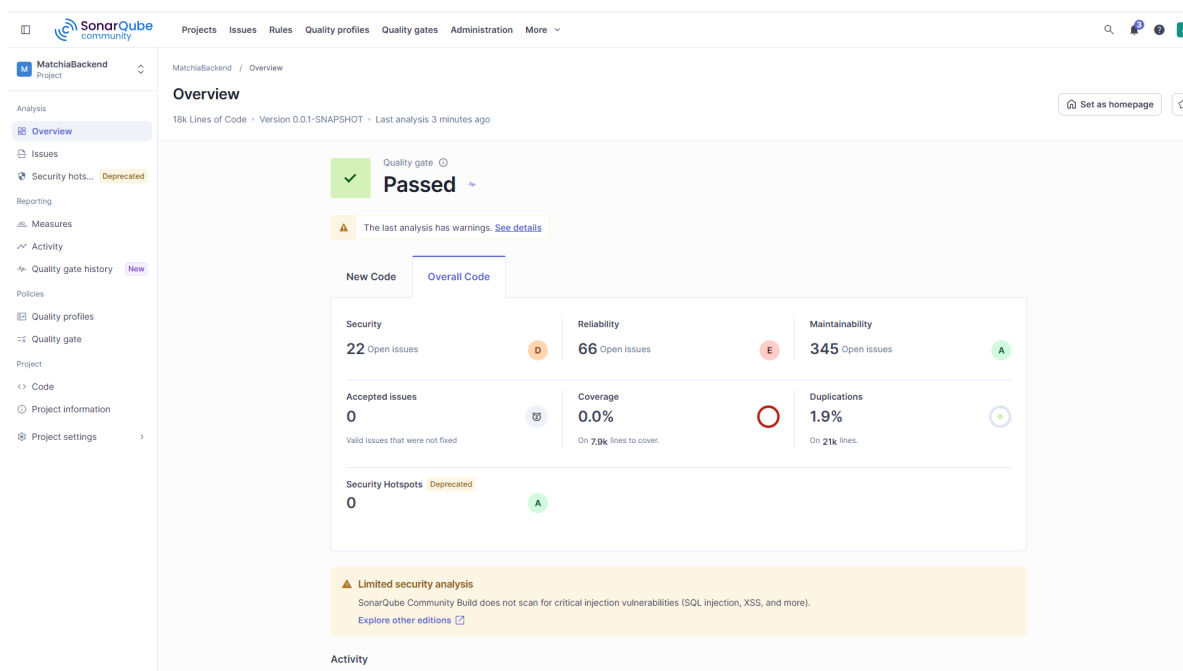


Figure 6.4 - Tableau de bord SonarQube du backend Matchia

Le Quality Gate apparaît comme validé, tandis que le tableau de bord expose les catégories de problèmes, la duplication et la couverture. La figure doit être accompagnée d'une interprétation factuelle : un Quality Gate vert ne dispense pas de corriger les problèmes restants ni d'améliorer la couverture affichée.

Capture exclue

La capture SonarFrontend.png expose une valeur assimilable à un jeton dans l'éditeur. Elle a été analysée comme preuve de configuration du scan frontend, mais elle ne doit pas être insérée telle quelle dans le mémoire. Utiliser une capture assainie ou le tableau de bord SonarQube correspondant.

6.11 Pipeline CI/CD global

Le pipeline global de Matchia peut être résumé de la manière suivante : développeur -> GitHub -> Jenkins -> checkout -> build backend et frontend -> tests backend -> scans SonarQube backend et frontend -> Docker build -> Docker Hub -> déploiement backend Azure -> déploiement frontend Azure. Cette représentation doit respecter l'ordre réellement codé. Les deux scans s'exécutent après les tests backend ; le Quality Gate est visible dans l'outillage mais le script ne contient pas de mécanisme de blocage explicite ; les tests frontend sont disponibles dans le projet mais non déclenchés dans ce Jenkinsfile.

Entrée	Contrôle	Sortie
Commit/push GitHub	Webhook githubPush et checkout SCM	Révision source associée à un numéro de build Jenkins.
Code backend	Maven, tests, JaCoCo, Sonar Maven	JAR vérifié et mesures de qualité disponibles.
Code frontend	npm ci, tsc -b, Vite, Sonar scanner	Assets dist et mesures de qualité disponibles.
Artefacts	Docker build et tags build/latest	Images frontend/backend sur Docker Hub.
Images	az containerapp update	Nouvelles révisions frontend et backend sur Azure Container Apps.

Tableau 6.3 - Chaîne CI/CD de Matchia

6.12 Déploiement sur Azure

Le Jenkinsfile configure une URL backend sous le domaine Azure Container Apps et exécute, pour chaque composant, la commande `az containerapp update`. Les images sont tirées de Docker Hub et portent à la fois un tag correspondant au numéro de build et un tag latest. Le déploiement précise le groupe de ressources `rg-matchia` et ajoute un suffixe de révision `v$BUILD_NUMBER`. Cette pratique permet d'identifier la révision livrée et de distinguer les mises à jour successives des Container Apps.

Le dépôt met donc en évidence Azure Container Apps pour le frontend et le backend, ainsi que Docker Hub comme registry. Il ne permet pas d'affirmer que les images sont stockées dans Azure Container Registry, car aucune commande ou configuration ACR n'est présente. Le fichier de procédure de redéploiement mentionne une base PostgreSQL Azure et la remise en service des ingress des deux Container Apps ; cette information peut être présentée comme procédure d'exploitation fournie avec le projet. En revanche, la capture `azure.png` ne montre qu'une page d'accueil du portail, sans ressource Matchia identifiable : elle ne doit pas être utilisée comme preuve du déploiement.

La preuve applicative la plus utile du déploiement est la consultation d'une marketplace tenant via l'URL Azure Container Apps. La capture suivante montre la marketplace `test1234` chargée avec son paramètre de tenant.

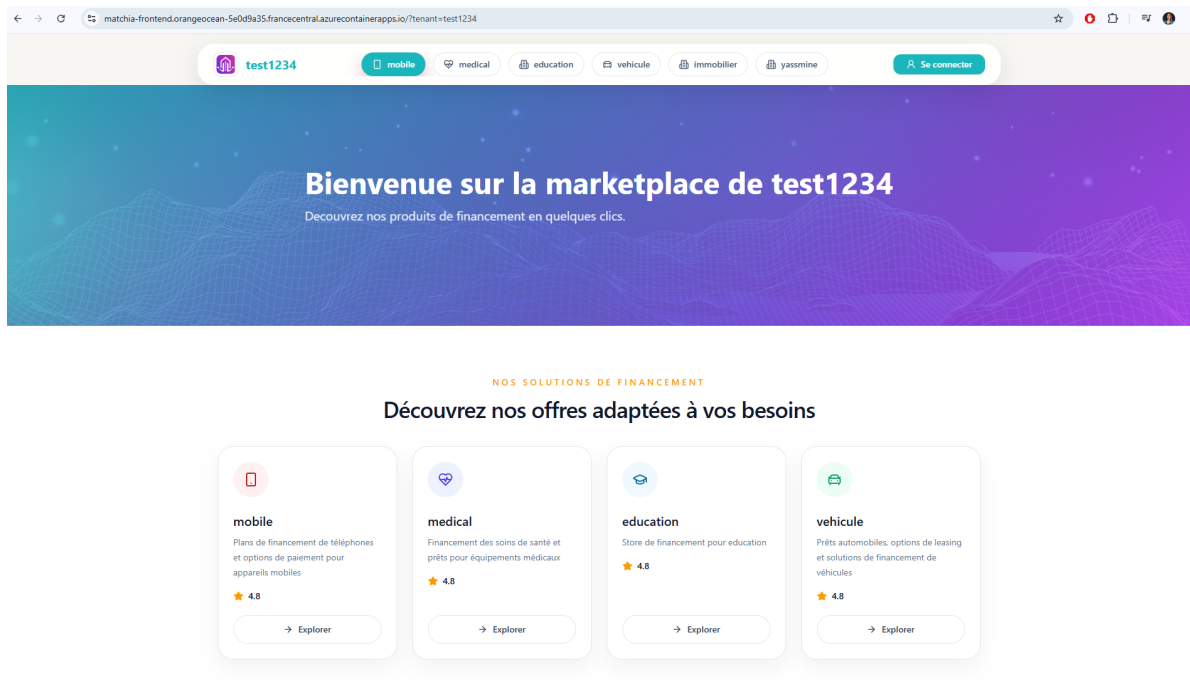


Figure 6.5 - Marketplace tenant déployée sur Azure Container Apps

Cette figure démontre que le frontend déployé restitue une expérience de marketplace configurée par tenant. Elle complète la capture Jenkins : la première atteste le déroulement du pipeline, tandis que celle-ci atteste le résultat applicatif accessible après déploiement.

6.13 Architecture de déploiement

L'architecture finale peut être représentée par le chemin suivant : utilisateur -> frontend React servi par Nginx dans une Azure Container App -> API Spring Boot dans une Azure Container App -> PostgreSQL. Le pipeline Jenkins construit les images, les publie dans Docker Hub et met à jour les deux Container Apps. Stripe, Gemini et SMTP restent des services externes appelés uniquement par le backend. Les uploads doivent être associés à un stockage persistant ; en local, le Compose monte un volume vers le répertoire uploads. Pour une exploitation cloud durable, le stockage, la sauvegarde et la politique de conservation doivent être explicitement configurés et documentés.

Figure à produire dans le mémoire

Dessiner un diagramme de déploiement avec les éléments vérifiés : GitHub, Jenkins, SonarQube, Docker Hub, Azure Container App Frontend, Azure Container App Backend, PostgreSQL, Stripe, Gemini, SMTP et stockage des uploads. Ne représenter Azure Container Registry qu'en cas de migration effective vers ce service.

6.14 Cycle de vie des ressources Azure

Le fichier Redéploiement.txt documente une procédure opérationnelle : redémarrage de PostgreSQL Azure, remise à disposition des ingress des Container Apps backend et frontend, reconnexion Azure CLI dans Jenkins si la session a expiré, puis relance du tunnel temporaire et vérification du webhook GitHub. Cette procédure traduit la nécessité de gérer l'état des ressources entre deux périodes de travail, notamment lorsque certaines ressources sont arrêtées pour limiter la consommation.

Le redéploiement applicatif reste centré sur la chaîne CI/CD : modification du code, push GitHub, lancement Jenkins, construction, publication d'une nouvelle image puis mise à jour de la Container App. Le suffixe de

révision utilisé par Jenkins facilite le suivi des versions. Avant une mise en production élargie, il est recommandé de formaliser une stratégie de retour arrière fondée sur les révisions Container Apps, un contrôle de santé applicatif, des sauvegardes de la base et une supervision centralisée.

6.15 Variables d'environnement et secrets

Les configurations Spring, Docker et Vite utilisent des variables d'environnement pour l'URL de la base, les identifiants, l'URL de l'API, les URLs publiques, les paramètres Stripe, la clé Gemini, le secret JWT, la messagerie et les répertoires de fichiers. Cette externalisation est indispensable pour distinguer les environnements local, CI et cloud. Dans le pipeline, les identifiants Docker Hub et le token Sonar sont récupérés via les mécanismes de credentials Jenkins, ce qui évite de les inscrire directement dans le Jenkinsfile.

La configuration actuellement visible contient également des valeurs de développement et certains artefacts de travail peuvent exposer des informations sensibles. Ces valeurs ne doivent jamais apparaître dans le mémoire. La cible de production doit privilégier des secrets injectés par Azure Container Apps ou un coffre-fort tel qu'Azure Key Vault, une rotation des clés, une base sans mot de passe par défaut, ainsi que la désactivation des logs de débogage inutiles. Cette recommandation est particulièrement importante pour une solution manipulant des données bancaires, des dossiers et des documents clients.

6.16 Sélection et interprétation des captures

Le dossier figure a été parcouru. Les captures suivantes sont retenues car elles apportent une preuve distincte et académique sans redondance. Les captures de tokens, clés API, mots de passe, configuration détaillée de connexion ou portail Azure générique doivent être écartées ou intégralement anonymisées.

Figure	Fichier source	Emplacement et rôle
Figure 6.1	imagesdocker.png	Section 6.4.2 ; confirme la construction des images frontend et backend.
Figure 6.2	containerdocker .png	Section 6.5 ; montre le démarrage des deux conteneurs applicatifs.
Figure 6.3	jenkins1.png	Section 6.6 ; visualise l'exécution complète du pipeline et ses stages.
Figure 6.4	SonarBackend.png	Section 6.10 ; présente un tableau de bord de qualité backend et le Quality Gate.
Figure 6.5	tenantDéploy.png	Section 6.12 ; démontre le résultat accessible du déploiement tenant sur Azure.
À exclure/assainir	SonarFrontend.png, dockercompose.png, clés/tokens	Ces captures peuvent contenir un jeton, un mot de passe ou une valeur d'environnement. Les anonymiser avant toute utilisation.
À exclure comme preuve	azure.png	Le portail Azure affiché ne montre pas une ressource Matchia identifiable ; il ne démontre pas le déploiement.

Tableau 6.4 - Captures DevOps retenues et règles d'utilisation

6.17 Difficultés rencontrées et solutions apportées

Les fichiers disponibles permettent d'identifier des difficultés de configuration sans en faire un journal d'erreurs. Premièrement, l'environnement comporte plusieurs services locaux - application, Jenkins, SonarQube et PostgreSQL - qui requièrent des réseaux, volumes et dépendances distincts. La solution mise en place est la séparation en fichiers Compose dédiés et l'usage de Docker-in-Docker pour permettre à Jenkins de construire et de publier les images. Deuxièmement, le frontend doit connaître l'URL du backend au moment de sa compilation Vite. Le pipeline injecte donc `AZURE_BACKEND_URL` par argument de build, ce qui évite de livrer un frontend pointant vers l'environnement local.

Troisièmement, la qualité de deux écosystèmes technologiques différents doit être consolidée. Le projet utilise JaCoCo et le plugin Maven pour le backend, et lcov avec `@sonar/scan` pour le frontend. Enfin, le déploiement cloud peut nécessiter une reconnexion de la CLI Azure et la remise en service de ressources arrêtées. La procédure de redéploiement documente ces opérations. Les améliorations recommandées sont l'automatisation du contrôle Quality Gate, l'exécution des tests frontend dans Jenkins, la centralisation des secrets et l'ajout de contrôles de santé et de rollback.

6.18 Bilan DevOps

La chaîne DevOps de Matchia apporte de la reproductibilité à un projet composé de deux applications et de plusieurs services externes. Docker garantit une construction homogène du backend et du frontend ; Docker Compose facilite l'exécution locale ; Jenkins relie la version GitHub à un pipeline de build, test, analyse, image et déploiement ; SonarQube rend visibles des indicateurs de qualité ; Azure Container Apps permet de livrer les services sous forme de révisions. Cette chaîne réduit les opérations manuelles et constitue une base solide pour l'exploitation d'une plateforme SaaS.

Son niveau de maturité doit toutefois être présenté avec précision. L'automatisation de l'analyse SonarQube est en place, mais le verrou Quality Gate explicite et les tests frontend dans le Jenkinsfile restent à renforcer. Les secrets et les données de connexion doivent être retirés des configurations locales et des captures. Ces améliorations ne remettent pas en cause les éléments réalisés ; elles définissent la trajectoire nécessaire pour rapprocher le prototype d'une exploitation de production soumise à des contraintes bancaires.

6.19 Conclusion

Ce chapitre a présenté les mécanismes d'industrialisation réellement implémentés dans Matchia. La conteneurisation sépare les couches frontend et backend, l'orchestration locale structure leur exécution, Jenkins automatise la livraison, SonarQube fournit un retour sur la qualité et Azure Container Apps reçoit les nouvelles révisions. La démarche DevOps prolonge ainsi l'architecture SaaS : elle rend l'évolution de la plateforme plus reproductible, plus traçable et plus facile à déployer pour l'ensemble des tenants bancaires.